

USER ACCEPTANCE TESTING (UAT)

Date	30 September 2026
Team ID	SWTID-2026-3024
Project Name	AI-Augmented Backend Development AI BlogNest API
Maximum Marks	

1. PROJECT OVERVIEW

Project Name: AI BlogNest – AI-Powered Blog Content Management System

Project Description:

AI BlogNest is an AI-powered backend system developed to simplify blog content creation and management. The system provides RESTful APIs for user authentication, role-based access, blog management, comments, database operations, AI-assisted blog generation, and content summarization.

Project Version: 1.0.0

2. TESTING SCOPE

The following features and functionalities are included in User Acceptance Testing:

- User registration
- User login and authentication
- JWT-based authentication
- Role-based access control
- Blog creation
- Blog retrieval
- Blog updating
- Blog deletion
- Comment management
- AI-powered blog content generation
- AI content summarization
- MongoDB database operations
- Input validation and sanitization
- Error handling
- API response verification

User Stories / Requirements to be Tested

- As a user, I should be able to register an account.
- As a registered user, I should be able to log in securely.

- As an authorized user, I should be able to create and manage blog posts.
- As a user, I should be able to retrieve available blog content.
- As an authorized user, I should only access operations permitted for my role.
- As a content creator, I should be able to generate blog content using AI.
- As a user, I should be able to obtain a concise summary of existing content.
- The system should securely store user and blog information in MongoDB.
- The system should return appropriate responses for valid and invalid requests.

3. TESTING ENVIRONMENT

URL/Location: Local development server – `http://localhost:5000`

API Testing Tool: ThunderClient

Backend Environment: Node.js with Express.js

Database: MongoDB

Database ODM: Mongoose

AI Service: Configured AI service / Google Gemini AI

Credentials: Authentication credentials are provided through the API during testing. Passwords are not stored in plain text.

4. TEST CASES

The following test cases were prepared to verify the functional, security, database, and AI-related features of the AI BlogNest backend system. Both positive and negative scenarios are included.

Test					
Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Pass/Fail
TC-001	User Registration with Valid Details	1. Open Postman. 2. Select the user registration API. 3. Enter valid name, email, password, and required details. 4. Send the POST request.	The system should validate the details, create a new user account, store the user information in MongoDB, and return a successful response.	User account was created successfully and a success response was received.	Pass

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Pass/Fail
TC-002	User Registration with Missing Required Fields	1. Open the registration API. 2. Leave one or more required fields empty. 3. Send the request.	The system should reject the request and return an appropriate validation error instead of creating the account.	The request was rejected and the required field validation message was returned.	Pass
TC-003	User Registration with Duplicate Email	1. Register a user with a valid email. 2. Try registering another user using the same email address. 3. Send the request.	The system should prevent duplicate registration and return an appropriate error message.	Duplicate registration was rejected by the system.	Pass
TC-004	User Login with Valid Credentials	1. Open the login API. 2. Enter a registered email and correct password. 3. Send the request.	The user should be authenticated successfully and the required authentication information/token should be returned.	Login was successful and the authentication token was returned.	Pass
TC-005	User Login with Invalid Credentials	1. Open the login API. 2. Enter an incorrect password or email. 3. Send the request.	The system should reject the login attempt and return an appropriate authentication error.	Invalid credentials were rejected and an error response was returned.	Pass
TC-006	Protected API Without Authentication	1. Select a protected API endpoint. 2. Do not provide a valid JWT token. 3. Send the request.	The system should deny access and return an unauthorized authentication response.	Access was denied without a valid authentication token.	Pass
TC-007	Protected API with Valid JWT	1. Log in successfully. 2. Copy the generated JWT token. 3. Add the token to the authorization header. 4. Access a protected endpoint.	The system should verify the token and allow the authenticated user to access the permitted endpoint.	Valid token was accepted and the protected endpoint was accessed successfully.	Pass
TC-008	Role-Based Authorization	1. Log in using a user with a specific role. 2. Access an endpoint requiring	The system should allow only the operations permitted for the user's assigned role and reject	Role-based access was applied and unauthorized operations were restricted.	Pass

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Pass/Fail
		authorization. 3. Send the request.	unauthorized operations.		
TC-009	Create Blog with Valid Data	1. Authenticate the user. 2. Open the blog creation endpoint. 3. Enter title, content, category, tags, and other required details. 4. Send the POST request.	A new blog post should be created and stored successfully in MongoDB.	Blog post was created and stored successfully.	Pass
TC-010	Create Blog with Invalid Data	1. Authenticate the user. 2. Open the blog creation endpoint. 3. Remove required information or provide invalid data. 4. Send the request.	The system should validate the submitted data and return an appropriate error without creating an invalid blog record.	Invalid request was rejected and a validation response was returned.	Pass
TC-011	Retrieve Blog Posts	1. Open the blog retrieval endpoint. 2. Send a GET request. 3. Review the returned response.	The system should retrieve available blog posts and return them in a structured response.	Blog records were retrieved successfully.	Pass
TC-012	Update Existing Blog	1. Authenticate the user. 2. Select an existing blog post. 3. Modify the required information. 4. Send the update request.	The selected blog post should be updated successfully and the modified information should be stored in MongoDB.	Blog information was updated successfully.	Pass
TC-013	Delete Existing Blog	1. Authenticate the user. 2. Select an existing blog post. 3. Send the delete request. 4. Verify the record.	The selected blog post should be deleted successfully and should no longer be available through the relevant retrieval operation.	Blog post was deleted successfully.	Pass
TC-014	Add Comment to Blog	1. Select an existing blog post. 2. Authenticate the user if required. 3. Enter a valid comment. 4.	The comment should be processed and associated with the selected blog post.	Comment was added successfully to the selected blog.	Pass

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Pass/Fail
		Send the comment request.			
TC-015	AI Blog Content Generation	1. Open the AI content generation endpoint. 2. Enter a suitable topic or prompt. 3. Send the request. 4. Review the response.	The AI service should process the prompt and return relevant blog content in the API response.	AI-generated blog content was returned successfully.	Pass
TC-016	AI Content Generation with Empty Prompt	1. Open the AI generation endpoint. 2. Leave the prompt empty. 3. Send the request.	The system should validate the input and return an appropriate error instead of processing an empty request.	Empty prompt was rejected with an appropriate validation response.	Pass
TC-017	AI Content Summarization	1. Open the content summarization endpoint. 2. Provide valid blog content. 3. Send the request. 4. Review the response.	The system should generate and return a concise summary of the provided content.	A summarized version of the provided content was returned successfully.	Pass
TC-018	MongoDB Data Storage Verification	1. Perform a valid create operation through the API. 2. Open the MongoDB database. 3. Check the corresponding collection.	The submitted information should be stored correctly in the appropriate MongoDB collection.	The created record was found in the database with the expected information.	Pass
TC-019	Invalid API Endpoint	1. Enter an incorrect or unavailable API URL. 2. Send the request through Postman.	The server should return an appropriate error response indicating that the requested resource or route was not found.	An appropriate error response was received for the invalid endpoint.	Pass
TC-020	Server Error and Response Handling	1. Send a request with invalid or unexpected input. 2. Observe the server response. 3. Check whether the API	The system should handle the error without crashing and return an appropriate error response to the client.	The error was handled and an appropriate API response was returned.	Pass

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Pass/Fail
		returns a structured response.			

Test Case Result Summary

Category	Number of Test Cases
Authentication & Authorization	7
Blog Management	5
Comments	1
AI Features	3
Database	1
API & Error Handling	3
Total	20

Overall Testing Result

The test cases cover the major functional requirements of AI BlogNest, including positive and negative scenarios. The authentication, authorization, blog management, comment management, AI content generation, AI summarization, database operations, validation, and error-handling functionalities were considered during testing.

The recorded test results indicate that the tested functionalities behaved according to their expected requirements.

5. BUG TRACKING

“No critical bugs were identified during the testing period. All tested functionalities produced the expected results.”

6. SIGN-OFF

Tester Name: Padmashri V

Date: 23/09/2026

Signature: ____Padmashri.v____

7. TESTING SUMMARY

The User Acceptance Testing of AI BlogNest focuses on verifying the major functional requirements of the backend system. The testing covers authentication, authorization, blog management, comments, AI-assisted content generation, content summarization, database operations, input validation, and error handling.

The APIs are tested using Postman by sending valid and invalid requests and comparing the actual responses with the expected results. The test results are recorded for each test case, and identified issues are documented in the bug tracking section.

The UAT process helps confirm that the implemented system satisfies the defined functional requirements and provides the expected behavior for the intended users.